



GRUB

GRUB  (GRand Unified Bootloader) is a **multi-boot loader**. It is derived from **PUPA**  which was a research project to develop the replacement of what is now known as **GRUB Legacy**. The latter had become too difficult to maintain and GRUB was rewritten from scratch with the aim to provide modularity and portability [\[1\]](#) . The current GRUB is also referred to as GRUB 2 while GRUB Legacy corresponds to versions 0.9x.

Note: In the entire article `esp` denotes the mountpoint of the **EFI system partition** aka ESP.

Contents [\[hide\]](#)

- 1 BIOS systems
 - 1.1 GUID Partition Table (GPT) specific instructions
 - 1.2 Master Boot Record (MBR) specific instructions
 - 1.3 Installation
- 2 UEFI systems
 - 2.1 Installation
- 3 Configuration
 - 3.1 Generated grub.cfg
 - 3.1.1 Generate the main configuration file
 - 3.1.2 Detecting other operating systems
 - 3.1.2.1 MS Windows
 - 3.1.3 Additional arguments
 - 3.1.4 LVM
 - 3.1.5 RAID
 - 3.1.6 Encrypted /boot
 - 3.2 Custom grub.cfg
 - 3.2.1 Boot menu entry examples
 - 3.2.1.1 GRUB commands
 - 3.2.1.1.1 "Shutdown" menu entry
 - 3.2.1.1.2 "Restart" menu entry
 - 3.2.1.1.3 "Firmware setup" menu entry (UEFI only)
 - 3.2.1.2 EFI binaries
 - 3.2.1.2.1 UEFI Shell
 - 3.2.1.2.2 gdisk
 - 3.2.1.2.3 Chainloading a unified kernel image
 - 3.2.1.3 Dual-booting
 - 3.2.1.3.1 GNU/Linux
 - 3.2.1.3.2 Windows installed in UEFI/GPT mode

Related articles

[Arch boot process](#)[Master Boot Record](#)[GUID Partition Table](#)[Unified Extensible Firmware Interface](#)[GRUB Legacy](#)[GRUB/EFI examples](#)[GRUB/Tips and tricks](#)[Multiboot USB drive](#)

3.2.1.3.3 Windows installed in BIOS/MBR mode

3.2.1.4 Using labels

4 Using the command shell

4.1 Pager support

4.2 Using the command shell environment to boot operating systems

4.2.1 Chainloading a partition's VBR

4.2.2 Chainloading a disk's MBR or a partitionless disk's VBR

4.2.3 Chainloading Windows/Linux installed in UEFI mode

4.2.4 Normal loading

4.3 Using the rescue console

5 GRUB removal

6 Troubleshooting

6.1 Unsupported file systems

6.2 Intel BIOS not booting GPT

6.3 Enable debug messages

6.4 msdos-style error message

6.5 UEFI

6.5.1 Common installation errors

6.5.2 Create a GRUB entry in the firmware boot manager

6.5.3 Drop to rescue shell

6.5.4 GRUB UEFI not loaded

6.5.5 Default/fallback boot path

6.6 Invalid signature

6.7 Boot freezes

6.8 Arch not found from other OS

6.9 Warning when installing in chroot

6.10 GRUB loads slowly

6.11 error: unknown filesystem

6.12 grub-reboot not resetting

6.13 Old BTRFS prevents installation

6.14 Windows 8/10 not found

6.15 VirtualBox EFI mode

6.16 Device /dev/xxx not initialized in udev database even after waiting 10000000 microseconds

6.17 GRUB rescue and encrypted /boot

7 See also

BIOS systems

GUID Partition Table (GPT) specific instructions

On a BIOS/**GPT** configuration, a **BIOS boot partition** is required. GRUB embeds its `core.img` into this partition.

Note:

- Before attempting this method keep in mind that not all systems will be able to support this partitioning scheme. Read more on **Partitioning#GUID Partition Table**.
- The BIOS boot partition is only needed by GRUB on a BIOS/GPT setup. On a BIOS/MBR setup, GRUB uses the post-MBR gap for the embedding the `core.img`. On GPT, however, there is no guaranteed unused space before the first

partition.

- For **UEFI** systems this extra partition is not required, since no embedding of boot sectors takes place in that case. However, UEFI systems still require an **EFI system partition**.

Create a mebibyte partition (`+1M` with *fdisk* or *gdisk*) on the disk with no file system and with partition type GUID `21686148-6449-6E6F-744E-656564454649`.

- Select partition type `BIOS boot` for **fdisk**.
- Select partition type code `ef02` for **gdisk**.
- For **parted** set/activate the flag `bios_grub` on the partition.

This partition can be in any position order but has to be on the first 2 TiB of the disk. This partition needs to be created before GRUB installation. When the partition is ready, install the bootloader as per the instructions below.

The space before the first partition can also be used as the BIOS boot partition though it will be out of GPT alignment specification. Since the partition will not be regularly accessed performance issues can be disregarded, though some disk utilities will display a warning about it. In *fdisk* or *gdisk* create a new partition starting at sector 34 and spanning to 2047 and set the type. To have the viewable partitions begin at the base consider adding this partition last.

Master Boot Record (MBR) specific instructions

Usually the post-MBR gap (after the 512 byte **MBR** region and before the start of the first partition) in many MBR partitioned systems is 31 KiB when DOS compatibility cylinder alignment issues are satisfied in the partition table. However a post-MBR gap of about 1 to 2 MiB is recommended to provide sufficient room for embedding GRUB's `core.img` (**FS#24103**). It is advisable to use a partitioning tool that supports 1 MiB **partition alignment** to obtain this space as well as to satisfy other non-512-byte-sector issues (which are unrelated to embedding of `core.img`).

Installation

Install the **grub** package. (It will replace **grub-legacy**^{AUR} if that is already installed.) Then do:

```
# grub-install --target=i386-pc /dev/sdX
```

where `/dev/sdX` is the **disk (not a partition)** where GRUB is to be installed. For example `/dev/sda` or `/dev/nvme0n1`, or `/dev/mmcblk0`. See **Device file#Block device names** for a description of the block device naming scheme.

Now you must **generate the main configuration file**.

If you use **LVM** for your `/boot`, you can install GRUB on multiple physical disks.

Tip: See **GRUB/Tips and tricks#Alternative installation methods** for other ways to install GRUB, such as to a USB stick.

See **grub-install(8)** and **GRUB Manual** for more details on the `grub-install` command.

UEFI systems

Note:

- It is recommended to read and understand the **Unified Extensible Firmware Interface, Partitioning#GUID Partition Table** and **Arch boot process#Under UEFI** pages.
- When installing to use UEFI it is important to boot the installation media in UEFI mode, otherwise *efibootmgr* will not be able to add the GRUB UEFI boot entry. Installing to the **fallback boot path** will still work even in BIOS mode since

it does not touch the NVRAM.

- To boot from a disk using UEFI, an EFI system partition is required. Follow [EFI system partition#Check for an existing partition](#) to find out if you have one already, otherwise you need to create it.

Installation

Note:

- UEFI firmwares are not implemented consistently across manufacturers. The procedure described below is intended to work on a wide range of UEFI systems but those experiencing problems despite applying this method are encouraged to share detailed information, and if possible the workarounds found, for their hardware-specific case. A [GRUB/EFI examples](#) article has been provided for such cases.
- The section assumes you are installing GRUB for x86_64 systems. For IA32 (32-bit) UEFI systems (not to be confused with 32-bit CPUs), replace `x86_64-efi` with `ia32-efi` where appropriate.

First, **install** the packages `grub` and `efibootmgr`: `GRUB` is the bootloader while `efibootmgr` is used by the GRUB installation script to write boot entries to NVRAM.

Then follow the below steps to install GRUB:

1. **Mount the EFI system partition** and in the remainder of this section, substitute `esp` with its mount point.
2. Choose a bootloader identifier, here named `GRUB`. A directory of that name will be created in `esp/EFI/` to store the EFI binary and this is the name that will appear in the UEFI boot menu to identify the GRUB boot entry.
3. Execute the following command to install the GRUB EFI application `grubx64.efi` to `esp/EFI/GRUB/` and install its modules to `/boot/grub/x86_64-efi/`.

```
# grub-install --target=x86_64-efi --efi-directory=esp --bootloader-id=GRUB
```

After the above install completed the main GRUB directory is located at `/boot/grub/`. Note that `grub-install` also tries to **create an entry in the firmware boot manager**, named `GRUB` in the above example -- this will, however, fail if your boot entries are full; use `efibootmgr` to remove unnecessary entries.

Remember to **#Generate the main configuration file** after finalizing the configuration.

Tip: If you use the option `--removable` then GRUB will be installed to `esp/EFI/BOOT/BOOTX64.EFI` (or `esp/EFI/BOOT/BOOTIA32.EFI` for the `ia32-efi` target) and you will have the additional ability of being able to boot from the drive in case EFI variables are reset or you move the drive to another computer. Usually you can do this by selecting the drive itself similar to how you would using BIOS. If dual booting with Windows, be aware Windows usually places an EFI executable there, but its only purpose is to recreate the UEFI boot entry for Windows.

Note:

- `--efi-directory` and `--bootloader-id` are specific to GRUB UEFI, `--efi-directory` replaces `--root-directory` which is deprecated.
- You might note the absence of a `device_path` option (e.g.: `/dev/sda`) in the `grub-install` command. In fact any `device_path` provided will be ignored by the GRUB UEFI install script. Indeed, UEFI bootloaders do not use a MBR bootcode or partition boot sector at all.
- Make sure to run the `grub-install` command from the system in which GRUB will be installed as the boot loader. That means if you are booting from the live installation environment, you need to be inside the chroot when running `grub-install`. If for some reason it is necessary to run `grub-install` from outside of the installed system,

append the `--boot-directory=` option with the path to the mounted `/boot` directory, e.g
`--boot-directory=/mnt/boot` .

See [UEFI troubleshooting](#) in case of problems. Additionally see [GRUB/Tips and tricks#UEFI further reading](#).

Configuration

On an installed system, GRUB loads the `/boot/grub/grub.cfg` configuration file each boot. You can follow [#Generated grub.cfg](#) for using a tool, or [#Custom grub.cfg](#) for a manual creation.

Generated grub.cfg

This section only covers editing the `/etc/default/grub` configuration file. See [GRUB/Tips and tricks](#) for more information.

Note: Remember to always [generate the main configuration file](#) after making changes to `/etc/default/grub` and/or files in `/etc/grub.d/` .

Generate the main configuration file

After the installation, the main configuration file `/boot/grub/grub.cfg` needs to be generated. The generation process can be influenced by a variety of options in `/etc/default/grub` and scripts in `/etc/grub.d/` .

If you have not done additional configuration, the automatic generation will determine the root filesystem of the system to boot for the configuration file. For that to succeed it is important that the system is either booted or chrooted into.

Note:

- The default file path is `/boot/grub/grub.cfg` , not `/boot/grub/i386-pc/grub.cfg` .
- If you are trying to run `grub-mkconfig` in a [chroot](#) or [systemd-nspawn](#) container, you might notice that it does not work: `grub-probe: error: failed to get canonical path of /dev/sdaX` . In this case, try using [arch-chroot](#) as described in the [BBS post](#) .
- If you are installing GRUB in chroot environment using LVM and the `grub-mkconfig` hangs indefinitely, see [#Device /dev/xxx not initialized in udev database even after waiting 10000000 microseconds](#) .

Use the `grub-mkconfig` tool to generate `/boot/grub/grub.cfg` :

```
# grub-mkconfig -o /boot/grub/grub.cfg
```

By default the generation scripts automatically add menu entries for all installed Arch Linux [kernels](#) to the generated configuration.

Tip:

- After installing or removing a [kernel](#), you just need to re-run the above `grub-mkconfig` command.
- For tips on managing multiple GRUB entries, for example when using both [linux](#) and [linux-lts](#) kernels, see [GRUB/Tips and tricks#Multiple entries](#) .

To automatically add entries for other installed operating systems, see [#Detecting other operating systems](#) .

You can add additional custom menu entries by editing `/etc/grub.d/40_custom` and re-generating `/boot/grub/grub.cfg` . Or you can create `/boot/grub/custom.cfg` and add them there. Changes to

`/boot/grub/custom.cfg` do not require re-running *grub-mkconfig*, since `/etc/grub.d/41_custom` adds the necessary `source` statement to the generated configuration file.

Tip: `/etc/grub.d/40_custom` can be used as a template to create `/etc/grub.d/nn_custom`, where `nn` defines the precedence, indicating the order the script is executed. The order scripts are executed determine the placement in the GRUB boot menu. `nn` should be greater than `06` to ensure necessary scripts are executed first.

See [#Boot menu entry examples](#) for custom menu entry examples.

Detecting other operating systems

To have *grub-mkconfig* search for other installed systems and automatically add them to the menu, [install](#) the `os-prober` package and [mount](#) the partitions from which the other systems boot. Then re-run *grub-mkconfig*.

Note: Remember to mount the partitions each time you run *grub-mkconfig* in order to include the other operating systems every time.

Tip: You might also want GRUB to remember the last chosen boot entry, see [GRUB/Tips and tricks#Recall previous entry](#).

MS Windows

Often, partitions containing Windows will be automatically discovered by `os-prober`. However, NTFS partitions may not always be detected when mounted with the default Linux drivers. If GRUB is not detecting it, try installing [NTFS-3G](#) and remounting.

Encrypted Windows partitions may need to be decrypted before mounting. For BitLocker, this can be done with `dislocker`^{AUR}. This should be sufficient for `os-prober` to add the correct entry.

Additional arguments

To pass custom additional arguments to the Linux image, you can set the `GRUB_CMDLINE_LINUX` + `GRUB_CMDLINE_LINUX_DEFAULT` variables in `/etc/default/grub`. The two are appended to each other and passed to kernel when generating regular boot entries. For the *recovery* boot entry, only `GRUB_CMDLINE_LINUX` is used in the generation.

It is not necessary to use both, but can be useful. For example, you could use

`GRUB_CMDLINE_LINUX_DEFAULT="resume=UUID=uuid-of-swap-partition quiet"` where *uuid-of-swap-partition* is the **UUID** of your swap partition to enable resume after [hibernation](#). This would generate a recovery boot entry without the resume and without `quiet` suppressing kernel messages during a boot from that menu entry. Though, the other (regular) menu entries would have them as options.

By default *grub-mkconfig* determines the **UUID** of the root filesystem for the configuration. To disable this, uncomment `GRUB_DISABLE_LINUX_UUID=true`.

For generating the GRUB recovery entry you have to ensure that `GRUB_DISABLE_RECOVERY` is not set to `true` in `/etc/default/grub`.

See [Kernel parameters](#) for more info.

LVM



This article or section is a candidate for merging with [#Installation](#).



Notes: *grub-mkconfig* is capable of detecting that it needs the `lvm` module, specifying it in `GRUB_PRELOAD_MODULES` is not required. Move warning to [#Installation](#) & [#Installation_2](#) or

create a [Known issues section](#) and document it there. (Discuss in [Talk:GRUB#](#))

Warning: GRUB does not support thin-provisioned logical volumes.

If you use [LVM](#) for your `/boot` or `/` root partition, make sure that the `lvm` module is preloaded:

```
/etc/default/grub
```

```
GRUB_PRELOAD_MODULES="... lvm"
```

RAID



This article or section is a candidate for merging with [#Installation](#).



Notes: grub-mkconfig is capable of detecting that it needs the `mdraid09` and/or `mdraid1x` modules, specifying them in `GRUB_PRELOAD_MODULES` is not required. Summarize the double grub-install in a note and move it to [#Installation](#); move `set root` stuff to [#Custom grub.cfg](#).

(Discuss in [Talk:GRUB#](#))

GRUB provides convenient handling of [RAID](#) volumes. You need to load GRUB modules `mdraid09` or `mdraid1x` to allow you to address the volume natively:

```
/etc/default/grub
```

```
GRUB_PRELOAD_MODULES="... mdraid09 mdraid1x"
```

For example, `/dev/md0` becomes:

```
set root=(md/0)
```

whereas a partitioned RAID volume (e.g. `/dev/md0p1`) becomes:

```
set root=(md/0,1)
```

To install grub when using RAID1 as the `/boot` partition (or using `/boot` housed on a RAID1 root partition), on BIOS systems, simply run *grub-install* on both of the drives, such as:

```
# grub-install --target=i386-pc --debug /dev/sda
# grub-install --target=i386-pc --debug /dev/sdb
```

Where the RAID 1 array housing `/boot` is housed on `/dev/sda` and `/dev/sdb`.

Note: GRUB supports booting from [Btrfs](#) RAID 0/1/10, but *not* RAID 5/6. You may use [mdadm](#) for RAID 5/6, which is supported by GRUB.

Encrypted /boot

GRUB also has special support for booting with an encrypted `/boot`. This is done by unlocking a [LUKS](#) blockdevice in order to read its configuration and load any [initramfs](#) and [kernel](#) from it. This option tries to solve the issue of having an

unencrypted boot partition.

Note: `/boot` is **not** required to be kept in a separate partition; it may also stay under the system's root `/` directory tree.

Warning: GRUB only supports LUKS2 headers (but not with Argon2i or Argon2id PBKDFs) in git master; see [GRUB bug #55093](#). Make sure to specify `--type luks1` when creating the encrypted partition using `cryptsetup luksFormat`.

To enable this feature encrypt the partition with `/boot` residing on it using **LUKS** as normal. Then add the following option to `/etc/default/grub`:

```
/etc/default/grub  
  
GRUB_ENABLE_CRYPTODISK=y
```

This option is used by `grub-install` to generate the `grub core.img`, so make sure to **install grub** after modifying this option.

Without further changes you will be prompted twice for a passphrase: the first for GRUB to unlock the `/boot` mount point in early boot, the second to unlock the root filesystem itself as implemented by the `initramfs`. You can use a **keyfile** to avoid this.

Warning:

- If you want to **generate the main configuration file**, make sure that `/boot` is mounted.
- In order to perform system updates involving the `/boot` mount point, ensure that the encrypted `/boot` is unlocked and mounted before performing an update. With a separate `/boot` partition, this may be accomplished automatically on boot by using **crypttab** with a **keyfile**.

Note:

- If you use a special keymap, a default GRUB installation will not know it. This is relevant for how to enter the passphrase to unlock the LUKS blockdevice. See [GRUB/Tips and tricks#Manual configuration of core image for early boot](#).
- If you experience issues getting the prompt for a password to display (errors regarding `cryptouid`, `cryptodisk`, or "device not found"), try reinstalling GRUB and appending `--modules="part_gpt part_msdos"` to the end of your `grub-install` command.

Tip: You can use **pacman hooks** to automount your `/boot` when upgrades need to access related files.

Custom grub.cfg



This article or section needs expansion.



Reason: Add instructions on how to write a custom `/boot/grub/grub.cfg`. See [User:Eschwartz/Grub](#) for a proposed draft. (Discuss in [Talk:GRUB#Manually generate grub.cfg](#))

This section describes the manual creation of GRUB boot entries in `/boot/grub/grub.cfg` instead of relying on `grub-mkconfig`.

A basic GRUB config file uses the following options:

- `(hdX, Y)` is the partition `Y` on disk `X`, partition numbers starting at 1, disk numbers starting at 0

- `set default=N` is the default boot entry that is chosen after timeout for user action
- `set timeout=M` is the time *M* to wait in seconds for a user selection before default is booted
- `menuentry "title" {entry options}` is a boot entry titled `title`
- `set root=(hdX,Y)` sets the boot partition, where the kernel and GRUB modules are stored (boot need not be a separate partition, and may simply be a directory under the "root" partition (`/`)

Boot menu entry examples

Tip: These boot entries can also be used when using a `/boot/grub/grub.cfg` generated by `grub-mkconfig`. Add them to `/etc/grub.d/40_custom` and **re-generate the main configuration file** or add them to `/boot/grub/custom.cfg`.

For tips on managing multiple GRUB entries, for example when using both [linux](#) and [linux-lts](#) kernels, see [GRUB/Tips and tricks#Multiple entries](#).

For [Archiso](#) and [Archboot](#) boot menu entries see [Multiboot USB drive#Boot entries](#).

GRUB commands

"Shutdown" menu entry

```
menuentry "System shutdown" {
    echo "System shutting down..."
    halt
}
```

"Restart" menu entry

```
menuentry "System restart" {
    echo "System rebooting..."
    reboot
}
```

"Firmware setup" menu entry (UEFI only)

```
if [ ${grub_platform} == "efi" ]; then
    menuentry "Firmware setup" {
        fwsetup
    }
fi
```

EFI binaries

When launched in UEFI mode, GRUB can chainload other EFI binaries.

Tip: To show these menu entries only when GRUB is launched in UEFI mode, enclose them in the following `if` statement:

```
if [ ${grub_platform} == "efi" ]; then
    place UEFI-only menu entries here
fi
```

UEFI Shell

You can launch **UEFI Shell** by placing it in the root of the **EFI system partition** and adding this menu entry:

```
menuentry "UEFI Shell" {
    insmod fat
    insmod chain
    search --no-floppy --set=root --file /shellx64.efi
    chainloader /shellx64.efi
}
```

gdisk

Download the **gdisk EFI application** and copy `gdisk_x64.efi` to `esp/EFI/tools/`.

```
menuentry "gdisk" {
    insmod fat
    insmod chain
    search --no-floppy --set=root --file /EFI/tools/gdisk_x64.efi
    chainloader /EFI/tools/gdisk_x64.efi
}
```

Chainloading a unified kernel image

If you have a **unified kernel image** generated from following **Secure Boot** or other means, you can add it to the boot menu. For example:

```
menuentry "Arch Linux" {
    insmod fat
    insmod chain
    search --no-floppy --set=root --fs-uuid FILESYSTEM_UUID
    chainloader /EFI/Linux/Arch-linux.efi
}
```

Dual-booting

GNU/Linux

Assuming that the other distribution is on partition `sda2`:

```
menuentry "Other Linux" {
    set root=(hd0,2)
    linux /boot/vmlinuz (add other options here as required)
    initrd /boot/initrd.img (if the other kernel uses/needs one)
}
```

Alternatively let GRUB search for the right partition by UUID or file system label:

```

menuentry "Other Linux" {
    # assuming that UUID is 763A-9CB6
    search --no-floppy --set=root --fs-uuid 763A-9CB6

    # search by label OTHER_LINUX (make sure that partition label is unambiguous)
    #search --no-floppy --set=root --label OTHER_LINUX

    linux /boot/vmlinuz (add other options here as required, for example: root=UUID
=763A-9CB6)
    initrd /boot/initrd.img (if the other kernel uses/needs one)
}

```

If the other distribution has already a valid `/boot` folder with installed GRUB, `grub.cfg`, kernel and initramfs, GRUB can be instructed to load these other `grub.cfg` files on-the-fly during boot. For example, for `hd0` and the fourth GPT partition:

```

menuentry "configfile hd0,gpt4" {
    insmod part_gpt
    insmod btrfs
    insmod ext2
    set root='hd0,gpt4'
    configfile /boot/grub/grub.cfg
}

```

When choosing this entry, GRUB loads the `grub.cfg` file from the other volume and displays that menu. Any environment variable changes made by the commands in file will not be preserved after `configfile` returns. Press `Esc` to return to the first GRUB menu.

Windows installed in UEFI/GPT mode

This mode determines where the Windows bootloader resides and chain-loads it after GRUB when the menu entry is selected. The main task here is finding the EFI system partition and running the bootloader from it.

Note: This menuentry will work only in UEFI boot mode and only if the Windows bitness matches the UEFI bitness. It will not work in BIOS installed GRUB. See [Dual boot with Windows#Windows UEFI vs BIOS limitations](#) and [Dual boot with Windows#Bootloader UEFI vs BIOS limitations](#) for more information.

```

if [ "${grub_platform}" == "efi" ]; then
    menuentry "Microsoft Windows Vista/7/8/8.1 UEFI/GPT" {
        insmod part_gpt
        insmod fat
        insmod chain
        search --no-floppy --fs-uuid --set=root $hints_string $fs_uuid
        chainloader /EFI/Microsoft/Boot/bootmgfw.efi
    }
fi

```

where `$hints_string` and `$fs_uuid` are obtained with the following two commands.

The `$fs_uuid` command determines the UUID of the EFI system partition:

```
# grub-probe --target=fs_uuid esp/EFI/Microsoft/Boot/bootmgfw.efi
```

```
1ce5-7f28
```

Alternatively one can run `blkid` (as root) and read the UUID of the EFI system partition from there.

The `$hints_string` command will determine the location of the EFI system partition, in this case harddrive 0:

```
# grub-probe --target=hints_string esp/EFI/Microsoft/Boot/bootmgfw.efi

--hint-bios=hd0,gpt1 --hint-efi=hd0,gpt1 --hint-baremetal=ahci0,gpt1
```

These two commands assume the ESP Windows uses is mounted at `esp`. There might be case differences in the path to Windows's EFI file, what with being Windows, and all.

Windows installed in BIOS/MBR mode

Note: GRUB supports booting `bootmgr` directly and [chainloading](#) of partition boot sector is no longer required to boot Windows in a BIOS/MBR setup.

Warning: It is the **system partition** that has `/bootmgr`, not your "real" Windows partition (usually `C:`). The system partition's **filesystem label** is `System Reserved` or `SYSTEM` and the partition is only about 100 to 549 MiB in size. See [Wikipedia:System partition and boot partition](#) for more information.

Throughout this section, it is assumed your Windows partition is `/dev/sda1`. A different partition will change every instance of `hd0,msdos1`.

Note: These menu entries will work only in BIOS boot mode. It will not work in UEFI installed GRUB. See [Dual boot with Windows#Windows UEFI vs BIOS limitations](#) and [Dual boot with Windows#Bootloader UEFI vs BIOS limitations](#).

In both examples `XXXXXXXXXXXXXXXXXX` is the filesystem UUID which can be found with command `lsblk --fs`.

For Windows Vista/7/8/8.1/10:

```
if [ "${grub_platform}" == "pc" ]; then
    menuentry "Microsoft Windows Vista/7/8/8.1/10 BIOS/MBR" {
        insmod part_msdos
        insmod ntfs
        insmod ntldr
        search --no-floppy --fs-uuid --set=root --hint-bios=hd0,msdos1 --hint-e
fi=hd0,msdos1 --hint-baremetal=ahci0,msdos1 XXXXXXXXXXXXXXXXXXXX
        ntldr /bootmgr
    }
fi
```

For Windows XP:

```
if [ "${grub_platform}" == "pc" ]; then
    menuentry "Microsoft Windows XP" {
        insmod part_msdos
        insmod ntfs
        insmod ntldr
        search --no-floppy --fs-uuid --set=root --hint-bios=hd0,msdos1 --hint-e
fi=hd0,msdos1 --hint-baremetal=ahci0,msdos1 XXXXXXXXXXXXXXXXXXXX
```

```
        ntldr /ntldr
    }
fi
```

Note: In some cases, GRUB may be installed without a clean Windows 8, in which case you cannot boot Windows without having an error with `\boot\bcd` (error code `0xc000000f`). You can fix it by going to Windows Recovery Console (`cmd.exe` from install disk) and executing:

```
X:\> bootrec.exe /fixboot
X:\> bootrec.exe /RebuildBcd
```

Do **not** use `bootrec.exe /Fixmbr` because it will wipe GRUB out. Or you can use Boot Repair function in the Troubleshooting menu - it will not wipe out GRUB but will fix most errors. Also you would better keep plugged in both the target hard drive and your bootable device **ONLY**. Windows usually fails to repair boot information if any other devices are connected.

Using labels

It is possible to use file system labels, human-readable strings attached to file systems, by using the `--label` option to `search`. First of all, **make sure your file system has a label**.

Then, add an entry using labels. An example of this:

```
menuentry "Arch Linux, session texte" {
    search --label --set=root archroot
    linux /boot/vmlinuz-linux root=/dev/disk/by-label/archroot ro
    initrd /boot/initramfs-linux.img
}
```

Using the command shell

Since the MBR is too small to store all GRUB modules, only the menu and a few basic commands reside there. The majority of GRUB functionality remains in modules in `/boot/grub/`, which are inserted as needed. In error conditions (e.g. if the partition layout changes) GRUB may fail to boot. When this happens, a command shell may appear.

GRUB offers multiple shells/prompts. If there is a problem reading the menu but the bootloader is able to find the disk, you will likely be dropped to the "normal" shell:

```
grub>
```

If there is a more serious problem (e.g. GRUB cannot find required files), you may instead be dropped to the "rescue" shell:

```
grub rescue>
```

The rescue shell is a restricted subset of the normal shell, offering much less functionality. If dumped to the rescue shell, first try inserting the "normal" module, then starting the "normal" shell:

```
grub rescue> set prefix=(hdX,Y)/boot/grub
grub rescue> insmod (hdX,Y)/boot/grub/i386-pc/normal.mod
rescue:grub> normal
```

Pager support

GRUB supports pager for reading commands that provide long output (like the `help` command). This works only in normal shell mode and not in rescue mode. To enable pager, in GRUB command shell type:

```
sh:grub> set pager=1
```

Using the command shell environment to boot operating systems

```
grub>
```

The GRUB's command shell environment can be used to boot operating systems. A common scenario may be to boot Windows / Linux stored on a drive/partition via **chainloading**.

Chainloading means to load another boot-loader from the current one, ie, chain-loading.

The other bootloader may be embedded at the start of a partitioned disk (MBR), at the start of a partition or a partitionless disk (VBR), or as an EFI binary in the case of UEFI.

Chainloading a partition's VBR

```
set root=(hdX,Y)
chainloader +1
boot
```

X=0,1,2... Y=1,2,3...

For example to chainload Windows stored in the first partition of the first hard disk,

```
set root=(hd0,1)
chainloader +1
boot
```

Similarly GRUB installed to a partition can be chainloaded.

Chainloading a disk's MBR or a partitionless disk's VBR

```
set root=hdX
chainloader +1
boot
```

Chainloading Windows/Linux installed in UEFI mode

```
insmod fat
set root=(hd0,gpt4)
chainloader (${root})/EFI/Microsoft/Boot/bootmgfw.efi
boot
```

`insmod fat` is used for loading the FAT file system module for accessing the Windows bootloader on the EFI system partition. `(hd0,gpt4)` or `/dev/sda4` is the EFI system partition in this example. The entry in the `chainloader` line specifies the path of the `.efi` file to be chain-loaded.

Normal loading

See the examples in [#Using the rescue console](#)

Using the rescue console

See [#Using the command shell](#) first. If unable to activate the standard shell, one possible solution is to boot using a live CD or some other rescue disk to correct configuration errors and reinstall GRUB. However, such a boot disk is not always available (nor necessary); the rescue console is surprisingly robust.

The available commands in GRUB rescue include `insmod`, `ls`, `set`, and `unset`. This example uses `set` and `insmod`. `set` modifies variables and `insmod` inserts new modules to add functionality.

Before starting, the user must know the location of their `/boot` partition (be it a separate partition, or a subdirectory under their root):

```
grub rescue> set prefix=(hdX,Y)/boot/grub
```

where `X` is the physical drive number and `Y` is the partition number.

Note: With a separate boot partition, omit `/boot` from the path (i.e. type `set prefix=(hdX,Y)/grub`).

To expand console capabilities, insert the `linux` module:

```
grub rescue> insmod i386-pc/linux.mod
```

or simply

```
grub rescue> insmod linux
```

This introduces the `linux` and `initrd` commands, which should be familiar.

An example, booting Arch Linux:

```
set root=(hd0,5)
linux /boot/vmlinuz-linux root=/dev/sda5
initrd /boot/initramfs-linux.img
boot
```

With a separate boot partition (e.g. when using UEFI), again change the lines accordingly:

Note: Since boot is a separate partition and not part of your root partition, you must address the boot partition manually, in the same way as for the prefix variable.

```
set root=(hd0,5)
linux (hdX,Y)/vmlinuz-linux root=/dev/sda6
initrd (hdX,Y)/initramfs-linux.img
boot
```

Note: If you experienced `error: premature end of file /YOUR_KERNEL_NAME` during execution of `linux` command, you can try `linux16` instead.

After successfully booting the Arch Linux installation, users can correct `grub.cfg` as needed and then reinstall GRUB.

To reinstall GRUB and fix the problem completely, changing `/dev/sda` if needed. See [#Installation](#) for details.

GRUB removal



This article or section needs expansion.



Reason: Migrating from BIOS booting to UEFI is not the only case where GRUB could be removed. Section needs to either cover how to remove GRUB installed for UEFI booting or it should be removed altogether as too trivial. (Discuss in [Talk:GRUB#](#))

After migrating to GPT/UEFI one may want to remove the [MBR boot code using dd](#):

```
# dd if=/dev/zero of=/dev/sdX bs=440 count=1
```

Troubleshooting

Unsupported file systems

In case that GRUB does not support the root file system, an alternative `/boot` partition with a supported file system must be created. In some cases, the development version of GRUB [grub-git](#)^{AUR} may have native support for the file system.

If GRUB is used with an unsupported file system it is not able to extract the **UUID** of your drive so it uses classic non-persistent `/dev/sdXx` names instead. In this case you might have to manually edit `/boot/grub/grub.cfg` and replace `root=/dev/sdXx` with `root=UUID=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX`. You can use the `blkid` command to get the UUID of your device, see [Persistent block device naming](#).

Intel BIOS not booting GPT



This article or section is a candidate for moving to [Arch boot process#Troubleshooting](#).



Notes: The issue is not limited to GRUB nor to just GPT. (Discuss in [Talk:GRUB#](#))



The factual accuracy of this article or section is disputed.



Reason: There would be no "1007 KiB partition" in a GPT disk's protective MBR unless hybrid-MBR was used and there is no reason to assume that it would. On GPT, by default, there is only one partition in the protective MBR—`0xEE`—which, although violates the specification, can be marked bootable using `fdisk` or `parted`. (Discuss in [Talk:GRUB#](#))

Some Intel BIOS's require at least one bootable MBR partition to be present at boot, causing GPT-partitioned boot setups to be unbootable.

This can be circumvented by using (for instance) `fdisk` to mark one of the GPT partitions (preferably the 1007 KiB partition you have created for GRUB already) bootable in the MBR. This can be achieved, using `fdisk`, by the following commands: Start `fdisk` against the disk you are installing, for instance `fdisk /dev/sda`, then press `a` and select the partition you wish to mark as bootable (probably #1) by pressing the corresponding number, finally press `w` to write the changes to the MBR.

Note: The bootable-marking must be done in `fdisk` or similar, not in GParted or others, as they will not set the bootable flag in the MBR.

With `cfdisk`, the steps are similar, just `cfdisk /dev/sda`, choose bootable (at the left) in the desired hard disk, and quit saving.

With recent version of `parted`, you can use `disk_toggle pmbr_boot` option. Afterwards verify that Disk Flags show `pmbr_boot`.

```
# parted /dev/sdx disk_toggle pmbr_boot
# parted /dev/sdx print
```

More information is available [here](#) 

Enable debug messages

Note: This change is overwritten when [#Generate the main configuration file](#).

Add:

```
set pager=1
set debug=all
```

to `grub.cfg`.

msdos-style error message

```
grub-setup: warn: This msdos-style partition label has no post-MBR gap; embedding will
not be possible!
grub-setup: warn: Embedding is not possible. GRUB can only be installed in this setup b
y using blocklists.
           However, blocklists are UNRELIABLE and its use is discouraged.
grub-setup: error: If you really want blocklists, use --force.
```

This error may occur when you try installing GRUB in a VMware container. Read more about it [here](#) . It happens when the first partition starts just after the MBR (block 63), without the usual space of 1 MiB (2048 blocks) before the first partition.

Read [#Master Boot Record \(MBR\) specific instructions](#)

UEFI

Common installation errors

- If you have a problem when running *grub-install* with *sysfs* or *procfs* and it says you must run `modprobe efivarfs`, try [Unified Extensible Firmware Interface#Mount efivarfs](#).
- Without `--target` or `--directory` option, *grub-install* cannot determine for which firmware to install. In such cases *grub-install* will print `source_dir does not exist. Please specify --target or --directory`.
- If after running *grub-install* you are told your partition does not look like an EFI partition then the partition is most likely not `Fat32`.

Create a GRUB entry in the firmware boot manager



This article or section needs expansion.

Reason: Provide an `efibootmgr` command. (Discuss in [Talk:GRUB#](#))



`grub-install` automatically tries to create a menu entry in the boot manager. If it does not, then see [UEFI#efibootmgr](#) for instructions to use `efibootmgr` to create a menu entry. However, the problem is likely to be that you have not booted your CD/USB in UEFI mode, as in [UEFI#Create UEFI bootable USB from ISO](#).

Drop to rescue shell

If GRUB loads but drops into the rescue shell with no errors, it can be due to one of these two reasons:

- It may be because of a missing or misplaced `grub.cfg`. This will happen if GRUB UEFI was installed with `--boot-directory` and `grub.cfg` is missing,
- It also happens if the boot partition, which is hardcoded into the `grubx64.efi` file, has changed.

GRUB UEFI not loaded

An example of a working UEFI:

```
# efibootmgr -v

BootCurrent: 0000
Timeout: 3 seconds
BootOrder: 0000,0001,0002
Boot0000* GRUB HD(1,800,32000,23532fbb-1bfa-4e46-851a-b494bfe9478c) File(\EFI\GRUB\grubx64.efi)
Boot0001* Shell HD(1,800,32000,23532fbb-1bfa-4e46-851a-b494bfe9478c) File(\shellx64.efi)
Boot0002* Festplatte BIOS(2,0,00)P0: SAMSUNG HD204UI
```

If the screen only goes black for a second and the next boot option is tried afterwards, according to [this post](#), moving GRUB to the partition root can help. The boot option has to be deleted and recreated afterwards. The entry for GRUB should look like this then:

```
Boot0000* GRUB HD(1,800,32000,23532fbb-1bfa-4e46-851a-b494bfe9478c) File(\grubx64.efi)
```

Default/fallback boot path

Some UEFI firmwares require a bootable file at a known location before they will show UEFI NVRAM boot entries. If this is the case, `grub-install` will claim `efibootmgr` has added an entry to boot GRUB, however the entry will not show up in the VisualBIOS boot order selector. The solution is to install GRUB at the default/fallback boot path:

```
# grub-install --target=x86_64-efi --efi-directory=esp --removable
```

Alternatively you can move an already installed GRUB EFI executable to the default/fallback path:

```
# mv esp/EFI/grub esp/EFI/BOOT
# mv esp/EFI/BOOT/grubx64.efi esp/EFI/BOOT/BOOTX64.EFI
```

Invalid signature

If trying to boot Windows results in an "invalid signature" error, e.g. after reconfiguring partitions or adding additional hard drives, (re)move GRUB's device configuration and let it reconfigure:

```
# mv /boot/grub/device.map /boot/grub/device.map-old
# grub-mkconfig -o /boot/grub/grub.cfg
```

`grub-mkconfig` should now mention all found boot options, including Windows. If it works, remove `/boot/grub/device.map-old`.

Boot freezes

If booting gets stuck without any error message after GRUB loading the kernel and the initial ramdisk, try removing the `add_efi_memmap` kernel parameter.

Arch not found from other OS

Some have reported that other distributions may have trouble finding Arch Linux automatically with `os-prober`. If this problem arises, it has been reported that detection can be improved with the presence of `/etc/lsb-release`. This file and updating tool is available with the package [lsb-release](#).

Warning when installing in chroot

When installing GRUB on a LVM system in a chroot environment (e.g. during system installation), you may receive warnings like

```
/run/lvm/lvmetad.socket: connect failed: No such file or directory
```

or

```
WARNING: failed to connect to lvmetad: No such file or directory. Falling back to internal scanning.
```

This is because `/run` is not available inside the chroot. These warnings will not prevent the system from booting, provided that everything has been done correctly, so you may continue with the installation.

GRUB loads slowly

GRUB can take a long time to load when disk space is low. Check if you have sufficient free disk space on your `/boot` or `/` partition when you are having problems.

error: unknown filesystem

GRUB may output `error: unknown filesystem` and refuse to boot for a few reasons. If you are certain that all **UUIDs** are correct and all filesystems are valid and supported, it may be because your **BIOS Boot Partition** is located outside the first 2 TiB of the drive [2]. Use a partitioning tool of your choice to ensure this partition is located fully within the first 2 TiB, then reinstall and reconfigure GRUB.

This error might also be caused by an **ext4** filesystem having the features `large_dir` or `metadata_csum_seed` set.

grub-reboot not resetting

GRUB seems to be unable to write to root BTRFS partitions [3]. If you use grub-reboot to boot into another entry it will therefore be unable to update its on-disk environment. Either run grub-reboot from the other entry (for example when switching between various distributions) or consider a different file system. You can reset a "sticky" entry by executing `grub-editenv create` and setting `GRUB_DEFAULT=0` in your `/etc/default/grub` (do not forget `grub-mkconfig -o /boot/grub/grub.cfg`).

Old BTRFS prevents installation

If a drive is formatted with BTRFS without creating a partition table (eg. `/dev/sdx`), then later has partition table written to, there are parts of the BTRFS format that persist. Most utilities and OS's do not see this, but GRUB will refuse to install, even with `--force`

```
# grub-install: warning: Attempting to install GRUB to a disk with multiple partition labels. This is not supported yet..
# grub-install: error: filesystem `btrfs' does not support blocklists.
```

You can zero the drive, but the easy solution that leaves your data alone is to erase the BTRFS superblock with `wipefs -o 0x10040 /dev/sdx`

Windows 8/10 not found

A setting in Windows 8/10 called "Hiberboot", "Hybrid Boot" or "Fast Boot" can prevent the Windows partition from being mounted, so `grub-mkconfig` will not find a Windows install. Disabling Hiberboot in Windows will allow it to be added to the GRUB menu.

VirtualBox EFI mode

For VirtualBox < 6.1, install GRUB to the **default/fallback boot path**.

See also **VirtualBox#Installation in EFI mode on VirtualBox < 6.1**.

Device /dev/xxx not initialized in udev database even after waiting 10000000 microseconds

If grub-mkconfig hangs and gives error:

```
WARNING: Device /dev/xxx not initialized in udev database even after waiting 10000000 microseconds
```

You may need to provide `/run/lvm/` access to the chroot environment using:

```
# mkdir /mnt/hostlvm
# mount --bind /run/lvm /mnt/hostlvm
# arch-chroot /mnt
# ln -s /hostlvm /run/lvm
```